

Problem A. Ascari's Record

Input file: *standard input*
Output file: *standard output*
Time limit: 2 seconds
Memory limit: 512 mebibytes

В сезоне 2002 года в Формуле-1 чемпион мира Макс Ферстаппен выиграл 15 гонок и 22, что является наибольшим количеством побед за сезн в истории. Однако это не лучшее относительное количество побед в истории: лидером здесь является Альберто Аскари, который в сезоне-1952 выиграл 6 гонок из 8, что даёт соотношение побед 0.75 — больше, чем $15/22$ у Макса.

Вам дано целое число — количество гонок в сезоне. Выведите одно целое число — минимальное количество гонок, в которых надо одержать победу для того, чтобы побить рекорд Аскари.

Input

Первая строка входных данных содержит одно целое число r ($7 \leq r \leq 50$) — количество гонок в сезоне.

Output

Выведите одно целое число — количество побед в сезоне с r гонками, требуемое для того, чтобы побить рекорд Аскари.

Examples

standard input	standard output
8	7
22	17
24	19

Problem B. Big Integer

Input file: *standard input*
Output file: *standard output*
Time limit: 2 seconds
Memory limit: 512 mebibytes

Дано целое положительное число, записанное в десятичной системе счисления без ведущих нулей.

Бекболат хочет поставить **ровно один** знак '+', '-' или '*' (обозначающий сложение, умножению и делению, соответственно) между двумя соседними цифрами в числе так, чтобы результат операции делился на 3 (заметим, что во втором операнде **могут** быть ведущие нули).

Каким количеством способов он это может сделать? Два способа считаются разными, если знак поставлен в разных местах или если знаки находятся в одном месте, но различаются.

Input

Первая строка входных данных содержит одно целое число N ($1 \leq N < 10^{100\,000}$).

Output

Выведите одно целое число — количество различных способов поставить один знак операции так, чтобы получилось число, кратное трём.

Examples

standard input	standard output
27	1
11	1
2022	3

Problem C. Chess Pieces Placement

Input file: *standard input*
Output file: *standard output*
Time limit: 2 seconds
Memory limit: 512 mebibytes

Это интерактивная задача

Где-то на шахматной доске расположена ровно одна некая белая шахматная фигура. Других фигур на доске пока нет. Вы не можете видеть доску (и фигуру), но ваша задача — поставить чёрного короля на безопасную клетку, используя не более трёх попыток.

Вам предстоит сыграть некоторое количество раундов. Количество раундов сообщается вам предварительно.

В каждом раунде вам даются три попытки на то, чтобы поставить короля. Вы выбираете координаты поля, если короля туда поставить нельзя (поле под боем или же занято самой белой фигурой), вам сообщается, что попытка неуспешна. Иначе вам сообщается об успехе попытки, после чего можно переходить к следующему раунду.

Если в каком-то раунде все три попытки будут неудачными, программа получит вердикт `Wrong Answer`.

Interaction Protocol

Взаимодействие начинается программа жюри, которая сообщает целое число T — количество раундов, которое предстоит сыграть ($1 \leq T \leq 16$).

В каждом раунде ваша программа отправляет на стандартный вывод предполагаемую позицию короля в обычной шахматной записи (вертикали пронумерованы буквами 'a' - 'h' слева направо, горизонтали пронумерованы последовательными целыми числами от 1 до 8 снизу вверх, и координаты поля задаются строкой из двух символов, например, "a3"). Если на это поле короля поставить нельзя, то программа жюри выведет число 0, после чего можно делать следующую попытку в раунде; если же короля поставить удалось, программа жюри выведет число 1 и текущий раунд закончится.

По окончании последнего раунда ваша программа должна корректно завершить выполнение.

В этой задаче интерактор **адаптивный**, то есть положение и тип фигуры могут определяться постфактум, но в соответствии с вашими запросами и полученными ответами.

Во избежание ошибки `Idleness Limit` не забывайте после каждого вывода позиции короля вывести ровно один символ перевода строки, а также сбросить буфер вывода, используя функцию `flush` вашего языка программирования: `fflush(stdout)`; для C/C++ (`stdio`), `std::cout.flush()`; для C++ (`iostream`), `System.out.flush()`; для Java и `sys.stdout.flush()` для Python.

Examples

standard input	standard output
3	
1	a2
0	e2
1	e4
0	a8
0	b8
0	c8
1	

Problem D. Discount

Input file: *standard input*
Output file: *standard output*
Time limit: 2 seconds
Memory limit: 512 mebibytes

В булочной “Бейкер-Стрит” — новая акция! Прайс-лист булочной содержит N строк; каждая строка содержит текст “ K пирогов стоят $F(K)$ ”, где $F(K) = \text{round}(K \cdot P)$, а P — цена одного пирога.

Функция *round* округляет вещественное число до ближайшего целого. В случае равенства (то есть 0.5 в дробной части) она округляет число вверх, например, $\text{round}(3141.5) = 3142$.

У вас есть прайс-лист и вам интересно узнать наименьшее возможное значение P .

Input

Первая строка входных данных содержит одно целое число N — количество строк в прайс-листе ($1 \leq N \leq 100$). Каждая из последующих N строк содержит по два целых числа K и $F(K)$ — количество пирогов и цена этого количества пирогов по акции ($1 \leq K \leq 100$, $0 \leq F(K) \leq 10^4$). Гарантируется, что существует как минимум одно такое значение P , при котором прайс-лист будет выглядеть соответствующим образом.

Output

Выведите одно целое число — наименьшее возможное значение P с абсолютной погрешностью не хуже 10^{-4} .

Examples

standard input	standard output
1 5 5	0.9
3 5 7 4 6 2 3	1.375

Problem E. Effective Transmission

Input file: *standard input*
Output file: *standard output*
Time limit: 2 seconds
Memory limit: 512 mebibytes

Как известно, без усилителя зона эффективного приёма сигнала от базовой станции мобильной связи является кругом. Энергорасход станции зависит от радиуса этого круга

Направленный усилитель может увеличить покрытие в p раз в определённом направлении без увеличения энергорасхода станции. Иначе говоря, зона покрытия становится эллипсом, а энергорасход зависит от длины его меньшей полуоси.

У вас есть базовая станция с уже установленным усилителем, который вы **не можете** поворачивать.

По заданным координатам абонентов на плоскости, расположите базовую станцию с усилителем так, чтобы все абоненты оказались в зоне эффективного приёма сигнала, а меньшая полуось эллипса зоны покрытия была как можно меньше.

Input

Первая строка входных данных содержит одно целое число n — количество абонентов ($1 \leq n \leq 5 \cdot 10^4$).

Каждая из последующих n строк содержит два целых числа x и y — координаты очередного абонента ($-2 \cdot 10^8 \leq x, y \leq 2 \cdot 10^8$).

Последующая строка содержит одно целое число a — направление усиления, то есть направление усиления повернуто на a градусов против часовой стрелки относительно положительного направления оси x ($0 \leq a < 180$).

Последняя строка содержит одно целое число p — коэффициент усиления ($1 \leq p \leq 100$).

Output

Выведите минимальную длину меньшей полуоси эллипса зоны покрытия с абсолютной или относительной ошибкой не хуже 10^{-3} .

Examples

standard input	standard output
2 2 0 -2 0 0 3	0.666666666666667

Problem F. Fix The Triangle

Input file: *standard input*
Output file: *standard output*
Time limit: 2 seconds
Memory limit: 512 mebibytes

Вам даны три палочки с целой длиной a , b and c . Ваша задача — найти такое целое число x , что треугольник со сторонами $a + x$, $b + x$ и $c + x$ существует и является прямоугольным. Заметим, что x может быть как положительным, так и отрицательным или нулём.

Если существует более одного x с таким свойством, выведите то, которое больше по абсолютной величине. Если неоднозначность сохраняется (два числа противоположных знаков), выведите положительное значение.

Input

Входные данные содержат строку с тремя целыми числами a , b и c ($1 \leq a, b, c \leq 10^4$).

Output

Если такого x не существует, выведите “Impossible” в первой строке. Иначе выведите “Possible” и во второй строке выведите значение x .

Examples

standard input	standard output
3 4 2	Possible 1
27 11 2022	Impossible

Problem G. Guess The Permutation

Input file: *standard input*
Output file: *standard output*
Time limit: 2 seconds
Memory limit: 512 mebibytes

Это интерактивная задача.

В этой задаче вы должны угадать скрытую перестановку P длины N .

Вы можете задавать запросы в виде `? a b` и получать на них в качестве ответа расстояние между a и b в перестановке, то есть значение $|i - j|$, где $P_i = a$ и $P_j = b$.

Ваша задача — найти перестановку (или перевёрнутую перестановку) не более, чем за 20 000 запросов.

Interaction Protocol

Взаимодействие начинает программа жюри, которая выводит строку, содержащую одно целое число N — длину перестановки ($1 \leq N \leq 10\,000$).

Затем вы можете задавать запросы. Чтобы отправить запрос, отправьте строку `? a b`, где a и b — два целых числа от 1 до N , после чего программа жюри выведет одно целое число — расстояние между a и b в перестановке.

Когда вы готовы вывести ответ, выведите строку `! P1 ... Pn` — угаданную перестановку. Ответ считается правильным, если выведенная Вами перестановка совпадёт с загаданной или с её перевёрнутым вариантом $P_n \dots P_1$.

Интерактор **не является адаптивным**, то есть перестановка генерируется до запуска интерактора и не изменяется в процессе взаимодействия.

Во избежание ошибки Idleness Limit не забывайте после каждого запроса или вывода ответа вывести ровно один символ перевода строки, а также сбросить буфер вывода, используя функцию `flush` вашего языка программирования: `fflush(stdout)`; для C/C++ (stdio), `std::cout.flush()`; для C++ (iostream), `System.out.flush()`; для Java и `sys.stdout.flush()` для Python.

Examples

standard input	standard output
3	? 1 2
2	? 3 2
1	? 1 3
1	! 1 3 2

Problem H. Hidden Graph

Input file: *standard input*
Output file: *standard output*
Time limit: 2 seconds
Memory limit: 512 mebibytes

Это интерактивная задача.

Ваша задача — найти кратчайший путь между вершинами p и q в связном неориентированном графе с N вершинами и рёбрами с целыми положительными весами.

Но вся информация, которая у вас есть вначале — это количество вершин и числа p и q — номера вершин, путь между которыми ищется.

Вы можете задать вопрос о кратчайшем расстоянии между двумя вершинами a и b . Ваша задача — восстановить какой-либо кратчайший путь между p и q , используя не более $5N$ запросов.

Interaction Protocol

В начале взаимодействия программа жюри выводит три целых числа: количество вершин N , номер стартовой вершины p и номер конечной вершины q ($1 \leq N \leq 300$, $1 \leq p, q \leq N$).

Затем вы можете задавать вопросы. Чтобы задать вопрос, выведите строку $? a b$, где a и b — номера двух вершин ($1 \leq a, b \leq N$), после чего программа жюри сообщит одно целое число — расстояние между a и b . Гарантируется, что ответ не превышает 10^9 .

Когда вы готовы сообщить ответ, выведите строку $! x_1 \dots x_k$ — список вершин в кратчайшем пути, который вы нашли, в порядке обхода, при этом должно выполняться $x_1 = p$ и $x_k = q$. Это действие не считается запросом и может быть выполнено только один раз.

Интерактор **не является адаптивным**, то есть граф генерируется до старта интерактора и не меняется в процессе работы.

Во избежание ошибки Idleness Limit не забывайте после каждого запроса или вывода ответа вывести ровно один символ перевода строки, а также сбросить буфер вывода, используя функцию `flush` вашего языка программирования: `fflush(stdout)`; для C/C++ (`stdio`), `std::cout.flush()`; для C++ (`iostream`), `System.out.flush()`; для Java и `sys.stdout.flush()` для Python.

Examples

standard input	standard output
4 1 3	
3	? 4 3
2	? 1 4
1	? 2 1
2	? 3 2
3	? 1 3
1	? 4 2
	! 1 2 3

Problem I. IVXLCDM-string

Input file: *tandard input*
Output file: *standard output*
Time limit: 1 second
Memory limit: 512 mebibytes

Вам дана строка S , состоящая из букв 'I', 'V', 'X', 'L', 'C', 'D' и 'M'. Ваша задача — посчитать количество различных разбиений строки на подстроки, которые являются корректными римскими числами.

Таблица из Википедии показывает способ записи римских чисел:

Individual decimal places	Тысячи	Сотни	Десятки	Единицы
1	M	C	X	I
2	MM	CC	XX	II
3	MMM	CCC	XXX	III
4		CD	XL	IV
5		D	L	V
6		DC	LX	VI
7		DCC	LXX	VII
8		DCCC	LXXX	VIII
9		CM	XC	IX

Заметим, что

- Числа 4, 9, 40, 90, 400 and 900 записываются в реверсивной нотации, где первый символ вычитается из второго (например, для 40 ("XL") 'X' (10) вычитается из 'L' (50)). Это **единственные** места, где реверсивная нотация используется.
- Число, содержащее несколько десятичных цифр, строится дописыванием римского эквивалента каждой цифры, от старшего разряда к младшему.
- Если в десятичном разряде стоит 0, никаких цифр в этом разряде в римском представлении не пишется.
- Наибольшее число, которое может быть представлено в римской системе счисления — это число 3,999 (MMMCMXCIX).

Два разбиения считаются различными, если существует как минимум одна пара соседних букв, которые принадлежат к разным частям в одном разбиении и к одной и той же части в другом.

Ответ может быть очень большим, поэтому выведите остаток от его деления на 998 244 353.

Input

Единственная строка входных данных содержит строку длины n ($1 \leq n \leq 100\,000$), состоящую из букв 'I', 'V', 'X', 'L', 'C', 'D' и 'M'.

Output

Выведите одно целое число — остаток от деления на 998 244 353 количества способов разбить данную строку на корректные римские числа.

Example

tandard input	standard output
XXVII	16
XI	2
MMXXII	32
XXVIIIXIMXXII	1280

Problem J. Jewels of Ice and Fire

Input file: *standard input*
Output file: *standard output*
Time limit: 2 seconds
Memory limit: 512 mebibytes

В компьютерной игре “Heroes of Spells and Magic” маг сражается с n монстрами. У i -го монстра hp_i хитпойнтов (НР).

У мага есть два магических камня. Камень Льда уменьшает НР выбранного монстра на 2 за один заряд, Камень Пламени уменьшает НР выбранного монстра на 3 за один заряд.

Когда у монстра НР становится равным нулю или менее, монстр считается убитым.

Пусть $f(i)$ — наименьшее количество зарядов Камня Пламени, требуемого для того, чтобы убить всех монстров, если не более i зарядов Камня Льда было использовано.

Вычислите остаток от деления суммы $f(0) + f(1) + \dots + f(x)$ для заданного x на 998 244 353.

Input

Первая строка входных данных содержит два целых числа n и x ($1 \leq n \leq 10^5$, $0 \leq x \leq 10^5$) — количество монстров и максимальное значение количества использованных зарядов Камня Льда при вычислении суммы, соответственно.

Вторая строка содержит n целых чисел $hp_1, hp_2, \dots, hp_n$ ($1 \leq hp_i \leq 10^4$).

Output

Выведите одно целое число — остаток от деления $f(0) + f(1) + \dots + f(x)$ на 998 244 353.

Examples

standard input	standard output
3 2 1 2 3	6
3 2 2 2 2	6

Problem K. K-Colored DAG

Input file: *standard input*
Output file: *standard output*
Time limit: 12 seconds
Memory limit: 512 mebibytes

Рассмотрим ориентированные ациклические графы на n занумерованных вершинах, обладающие следующими свойствами:

- Каждое ребро окрашено в один из k различных цветов.
- Для каждой вершины существует не более одного исходящего ребра.
- Количество входящих рёбер для каждой вершины должно находиться в заданном множестве A .

Требуется подсчитать остаток от деления количества различных таких графов на 998 244 353.

Два графа различны тогда и только тогда, когда существует направленное ребро из некоторой вершины a в некоторую вершину b , присутствующее ровно в одном графе или же присутствующее в обоих графах, но окрашенное в разные цвета.

Input

Первая строка входных данных содержит три целых числа n , q и a ($1 \leq n \leq 9 \cdot 10^8$, $1 \leq k \leq 10^7$, $1 \leq a \leq 4$). Вторая строка содержит a попарно различных чисел a_i ($0 \leq a_i \leq 3$) — элементы множества A (то есть значения допустимого количества входящих в вершину рёбер).

Output

Выведите остаток от деления количества различных графов, удовлетворяющих условию, на 998 244 353.

Examples

standard input	standard output
3 1 2 0 1	13

Problem L. Least Divisor

Input file: *standard input*
Output file: *standard output*
Time limit: 2 seconds
Memory limit: 512 mebibytes

Это интерактивная задача.

Программа жюри загадала целое число n между 1 и 10^6 . Вы не знаете этого числа, но можете задавать запросы следующего формата:

? x — прибавить к **текущему** значению n целое число x между 0 и 2022 включительно, и сообщить наименьший простой делитель нового значения числа n .

Заметим, что результаты всех операций накапливаются, то есть перед выполнением очередного запроса n равно первоначальному значению плюс сумма всех x в предыдущих запросах.

Ваша задача — угадать первоначальное значение n не более, чем за 60 попыток.

Interaction Protocol

Взаимодействие начинается ваша программа.

Для того, чтобы задать запрос, отправьте строку ? x , где $0 \leq x \leq 2022$.

После чего вы получите одно целое число — наименьший простой делитель полученного после данного запроса значения n .

Когда вы готовы вывести ответ, отправьте строку ! n — **первоначальное** значение n . Это действие не считается запросом. Ответ может быть выведен только один раз.

В этой задаче интерактор **не адаптивный**, то есть целое число n генерируется перед началом взаимодействия и не меняется в его процессе.

Во избежание ошибки Idleness Limit не забывайте после каждого запроса или вывода ответа вывести ровно один символ перевода строки, а также сбросить буфер вывода, используя функцию `flush` вашего языка программирования: `fflush(stdout)`; для C/C++ (`stdio`), `std::cout.flush()`; для C++ (`iostream`), `System.out.flush()`; для Java и `sys.stdout.flush()` для Python.

Examples

standard input	standard output
7	? 1
3	? 2
19	? 8
	! 2022

Note

Note that in the example the contestant did not have enough reason to tell the answer, but he was lucky. Please do not assume that your luck will be the same.

Problem M. Memory Test

Input file: *standard input*
Output file: *standard output*
Time limit: 3 seconds
Memory limit: 512 mebibytes

Тест памяти для компьютера работает следующим образом: вам заданы два параметра N и Q — длина массива и количество запросов, соответственно. Первоначально массив arr размера N инициализируется как $arr = [1, 2, \dots, N]$, то есть $arr[i] = i$ (нумерация элементов массива начинается с единицы).

Далее тест выполняет два типа запросов.

Запрос первого типа имеет вид $1\ u\ v\ l$. Для каждого i между 0 и $l-1$ включительно выполняется присваивание $arr[u+i] := arr[v+i]$.

Запрос второго типа имеет вид $2\ x$ и требует вывести значение $arr[x]$.

Считая, что на вашем компьютере с памятью всё в порядке, реализуйте данный тест.

Input

Первая строка входных данных содержит два целых числа N и Q ($1 \leq N \leq 10^6$, $1 \leq Q \leq 10^5$) — длину массива и количество запросов, соответственно.

Каждая из последующих Q строк содержит один запрос. Формат запроса описан в условии. Для запроса первого типа $u, v \geq 1$ и $u+l-1, v+l-1 \leq N$. Для запроса второго типа $1 \leq x \leq N$.

Output

Для каждого запроса второго типа выведите одно целое число с новой строки — ответ на соответствующий запрос.

Examples

standard input	standard output
6 4	5
1 1 5 1	2
2 1	
1 3 1 4	
2 6	

Note

После первого изменения массив из примера выглядит так: $[5, 2, 3, 4, 5, 6]$, а после второго — так: $[5, 2, 5, 2, 5, 2]$.